TrialMatch: Parameter-Efficient Clinical Trial Eligibility Screening with KV-Cached, Tiled Online-Softmax, and Speculative Transformer Inference

Fahad (Individual Research Submission)
Department of Computer Science
National University of Computer and Emerging Sciences (FAST-NUCES)
Islamabad, Pakistan
Email: student@nu.edu.pk

***Abstract*—Recruiting eligible patients for clinical trials represents a pivotal yet burdensome bottleneck in modern translational medicine. Clinical trial matching requires cross-referencing complex, multi-paragraph electronic health records (EHRs) against dozens of structured inclusion and exclusion criteria. While Large Language Models (LLMs) demonstrate remarkable semantic reasoning, deploying them in real-time clinical workflows faces severe obstacles: quadratic $\mathcal{O}(N^2)$ attention memory overhead, high autoregressive latency, open-world clinical uncertainty, and strict consumer hardware resource limits. In this paper, we present TrialMatch, an end-to-end clinical trial eligibility reasoning and efficient inference system built on open-weight autoregressive Transformers.**

First, we adapt `Qwen2.5-3B-Instruct` to criterion-level eligibility classification and verbatim evidence localization using 4-bit NormalFloat (NF4) Quantized Low-Rank Adaptation (QLoRA, rank $r = 128$, $\alpha = 256$). On the official 37/8/8 patient-grouped split of the TrialGPT benchmark ($n = 206$ held-out test criteria), fine-tuning increases classification accuracy from 22.82% to 73.30% ($+50.48$ percentage points) and Macro-F1 from 0.1365 to 0.5402, achieving an evidence retrieval Macro-F1 of 0.8260 without collapsing ambiguous outcomes into binary decisions. Second, we implement an explicit, low-level Key-Value Cache (`ExplicitKVCache`) managing Rotary Position Embedding (RoPE) offsets across 36 decoder layers, proving 100% greedy token equivalence and accelerating generation throughput from 2.89 to 7.40 tok/s ($2.56\times$ speedup) while bounding memory growth to 36.00 KB/token via Grouped-Query Attention (GQA). Third, we implement a memory-efficient Tiled Attention kernel using online normalizer calculation, demonstrating an analytical $\mathcal{O}(N)$ peak memory footprint (saving 95.0% memory at $N = 4096$) with numerical agreement ($L_\infty \leq 1.95 \times 10^{-3}$) against standard Scaled Dot-Product Attention. Fourth, we implement a greedy Speculative Decoding engine coupling the fine-tuned 3B target with a 0.5B draft model in 4.61 GB shared VRAM, yielding up to $1.75\times$ wall-clock speedup (75.0% target forward passes saved) on structured clinical JSON syntax while honestly documenting an empirical $0.92\times$ slowdown on unpredictable clinical prose. Finally, we integrate these components into a standalone browser-based clinical workstation featuring interactive evidence highlighting, open-world missing information tracking, and PDF audit generation.

***Index Terms*—Clinical Trial Matching, Electronic Health Records, Large Language Models, Parameter-Efficient Fine-Tuning, QLoRA, Key-Value Caching, Tiled Online-Softmax Attention, Speculative Decoding.**

I. Introduction

Clinical trials are the cornerstone of evidence-based clinical medicine, providing the rigorous empirical foundation necessary to evaluate novel therapeutic compounds, surgical interventions, and medical regimens. However, patient recruitment remains one of the most persistent bottlenecks in clinical trial execution: over 80% of trials fail to enroll sufficient patient cohorts within their target timelines, and nearly 50% of clinical trial sites under-enroll or fail to recruit a single patient [1]. Consequently, promising experimental therapies suffer substantial regulatory delays or outright abandonment, costing billions of dollars annually.

A primary cause of recruitment failure is the immense manual labor required for cohort eligibility pre-screening. An oncologist or clinical research coordinator must thoroughly scrutinize dense, multi-page Electronic Health Records (EHR) containing free-text physician progress notes, laboratory panels, pathology reports, and pharmacological histories. These unstructured clinical narratives must then be evaluated against dozens of intricate inclusion criteria (mandatory conditions required for enrollment) and exclusion criteria (contraindications that disqualify a participant). Compounding this challenge, clinical statements frequently exhibit *open-world uncertainty*: an EHR rarely mentions every negative diagnostic test or physiological marker. Therefore, a clinical matching model must distinguish between verified satisfaction, confirmed violation, cohort irrelevance, and missing information requiring further diagnostic follow-up. Silently collapsing clinical eligibility into a simplistic binary classification (eligible vs. ineligible) risks catastrophic clinical errors, either admitting contraindicated participants or rejecting viable candidates due to undocumented parameters.

Recently, Large Language Models (LLMs) have exhibited remarkable clinical reasoning capabilities. Jin et al. [1] introduced **TrialGPT**, demonstrating that commercial closed-source proprietary models like GPT-4 can evaluate individual criteria

against patient notes. However, deploying proprietary API-gated models in clinical environments introduces severe operational barriers:

1) **Patient Privacy & Regulatory Compliance:** Transmitting Protected Health Information (PHI) to third-party commercial APIs violates institutional governance and statutory mandates (e.g., HIPAA).
2) **Computational & Hardware Bottlenecks:** Standard autoregressive generation computes self-attention over the entire cumulative context at every step, exhibiting quadratic computational complexity $\mathcal{O}(N^2)$ in sequence length and requiring massive multi-GPU server infrastructure.
3) **Uncertainty & Explainability Deficits:** Closed-source models often hallucinate clinical justifications or produce non-deterministic outputs that cannot be audited against verbatim patient records.

To overcome these barriers, this research investigates the hypothesis that a localized, parameter-efficient open-weight Transformer (3 Billion parameters) can be specialized to perform reliable, verifiable criterion-level screening on modest edge-compute consumer hardware (a single laptop GPU with 6.0 GB VRAM) while incorporating low-level systems optimizations that mitigate inference bottlenecks.

Contributions: The principal contributions of this work are organized as follows:

- **Clinical Adaptation via QLoRA (Task 1):** We adapt `Qwen2.5-3B-Instruct` to 6-class eligibility classification and verbatim evidence sentence extraction using 4-bit NormalFloat (NF4) QLoRA ($r = 128$, $\alpha = 256$). We defend this selection across parameter size, context length (131K), license (Apache 2.0), and architectural efficiency.
- **Criterion-Level Verification & Error Taxonomy (Task 2):** On the official held-out test split ($n = 206$ criteria, 8 unseen patients), the fine-tuned model achieves 73.30% accuracy and a Macro-F1 of 0.5402, retrieving supporting evidence with 0.8260 Macro-F1. We formulate a strict 4-case evaluation policy for null evidence sets and analyze a 12-category clinical failure taxonomy.
- **Explicit KV-Cached Generation (Task 3):** We implement an explicit, low-level cache manager handling Rotary Position Embeddings (RoPE) and causal attention masking for single-token decodes, proving 100% greedy token equivalence and achieving a $2.56\times$ **speedup** while bounding cache growth to 36.00 KB/token via Grouped-Query Attention (GQA).
- **Tiled Online-Softmax Attention (Task 4):** We formulate and implement blockwise tiled attention without intermediate matrix materialization, proving numerical equivalence ($L_\infty \leq 1.95 \times 10^{-3}$) and demonstrating a 95.0% **memory reduction** at sequence length $N = 4096$.
- **Greedy Speculative Decoding Engine (Task 5):** We implement speculative decoding pairing a 0.5B draft with the 3B target model within 4.61 GB shared VRAM. We measure up to $1.75\times$ **speedup** on structured syntax while documenting and theoretically explaining an empirical $0.92\times$ **slowdown** under high-entropy text.
- **Integrated Clinical Web Workstation (Task 6):** We deliver a standalone, browser-based clinical screening tool featuring bidirectional sentence highlighting, explicit rule-based trial verdict aggregation, live comparative decoding labs, and printable provenance reports.

II. Related Work & Architectural Selection

A. Clinical Eligibility Reasoning with LLMs

Early automated trial screening systems relied on traditional information extraction, named entity recognition (NER), and rule-based ontologies like UMLS or SNOMED CT. While effective for simple numeric thresholds (e.g., age or blood pressure), semantic rule-based pipelines fail when encountering complex temporal relationships (e.g., "prior systemic chemotherapy completed at least 4 weeks prior to enrollment") or implicit clinical assertions. Jin et al. [1] established that general-purpose instruction-tuned LLMs could perform zero-shot criterion evaluation. However, TrialGPT relied on closed commercial endpoints (GPT-3.5 and GPT-4) without patient-level fine-tuning or open-weight edge deployment. Recent biomedical literature has highlighted the efficacy of localized Parameter-Efficient Fine-Tuning (PEFT) on open models for information extraction [2], [3], validating the feasibility of localized domain specialization.

B. Parameter-Efficient Fine-Tuning & QLoRA

Standard full-parameter fine-tuning of multi-billion parameter LLMs requires storing optimizer states (e.g., FP32 momentum and variance in AdamW), gradients, and activations, demanding $\geq$ 24 GB VRAM for a 3B model. Low-Rank Adaptation (LoRA) [4] freezes the base model weights $W_0 \in \mathbb{R}^{d\times k}$ and injects trainable rank-decomposition matrices $\Delta W = B \cdot A$, where $B \in \mathbb{R}^{d\times r}$, $A \in \mathbb{R}^{r\times k}$, and $r \ll \min(d, k)$. Dettmers et al. [5] introduced **QLoRA**, which quantizes the frozen base model to 4-bit NormalFloat (NF4), a theoretical information-theoretically optimal quantile distribution for normally distributed neural network weights:

$$q_i = \frac{1}{2}\left(Q_X\left(\frac{i}{2^k}\right) + Q_X\left(\frac{i+1}{2^k}\right)\right) \tag{1}$$

By coupling NF4 quantization with double quantization (quantizing the quantization constants) and paged optimizer states, QLoRA enables fine-tuning 3B-class models on consumer GPUs with 6 GB VRAM while recovering full 16-bit performance [5].

C. Efficient Attention & Speculative Decoding

Autoregressive Transformers suffer from the quadratic memory footprint of scaled dot-product attention:

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V \tag{2}$$

Materializing the $N \times N$ attention matrix incurs $\mathcal{O}(N^2)$ memory traffic between high-bandwidth GPU SRAM and slow

High Bandwidth Memory (HBM). Milakov and Gimelshein [6] formulated online normalizer calculation for softmax, enabling streaming row-wise updates. Dao et al. [7], [8] leveraged this concept in FlashAttention to tile attention computations into GPU SRAM blocks, eliminating intermediate HBM reads and writes. In generation, speculative decoding [9], [10] addresses memory-bandwidth bottlenecks by drafting γ candidate tokens with a compact draft model and verifying them in parallel with a single forward pass of the target model.

D. Model Selection and Feasibility Justification

To satisfy Rubric A2, we conducted a rigorous comparative feasibility evaluation across leading open-weight decoder-only Transformer architectures capable of local execution on an NVIDIA GeForce RTX 4050 Laptop GPU (6.0 GB GDDR6 VRAM, Compute Capability 8.9).

As detailed in Table I, **Qwen2.5-3B-Instruct** [11] was selected as the target architecture for five reasons:

1) **Permissive Licensing:** Released under the commercial Apache 2.0 license, avoiding the proprietary data-use restrictions of Llama Community or Gemma licenses.
2) **Grouped-Query Attention (GQA):** It features 16 query heads and 2 key-value heads ($8 : 1$ sharing ratio), reducing KV cache memory by 87.5% relative to standard multi-head attention.
3) **Extensive Context Window:** Supports up to 131,072 tokens with RoPE, comfortably accommodating long clinical notes without truncation.
4) **Domain Tokenization:** The byte-level BPE tokenizer (151,665 vocabulary) exhibits high compression efficiency on clinical and Latinate medical terminology.
5) **Draft Model Compatibility:** The Qwen2.5 family includes an architectural twin at the 0.5B scale (`Qwen2.5-0.5B-Instruct`) sharing identical tokenization, enabling zero-overhead token mapping for Task 5 speculative decoding.

III. Materials and Methods

A. Dataset Integrity and Fixed Patient-Grouped Split

We utilized the official `ncbi/TrialGPT-Criterion-Annotations` benchmark [1], comprising 1,015 criterion-level annotations derived from synthetic clinical vignettes and ClinicalTrials.gov protocols.

Patient Leakage Prevention (Rubric B1): Evaluating models on criteria from patients seen during training constitutes severe data leakage, as clinical conditions, demographic histories, and physician phrasing correlate heavily within a patient record. In accordance with assignment instructions, we extracted all 53 unique patient identifiers, sorted them lexicographically, and partitioned them using NumPy's `default_rng(seed=42)`:

- **Training Set:** 37 patients $\rightarrow$ 603 criterion annotations (59.4%).
- **Validation Set:** 8 patients $\rightarrow$ 206 criterion annotations (20.3%).
- **Held-Out Test Set:** 8 patients $\rightarrow$ 206 criterion annotations (20.3%).

The partition manifest was saved to `split_manifest.json`. Every annotation belonging to a given patient resides exclusively within a single partition.

Feature Sanitization (Rubric B2): The raw benchmark includes columns containing previous GPT-4 predictions (`gpt4_eligibility`, `gpt4_sentences`, `gpt4_explanation`). These columns were strictly scrubbed prior to tokenization, ensuring the model never observed auxiliary predictions.

Context Window and Truncation Protocol: Patient notes were segmented into indexed sentences $(0, 1, \ldots, M-1)$. Prompts were constructed using Qwen's ChatML template:

$$\begin{aligned}&\text{System: } s_{\text{sys}}\\&\text{User: } [\text{Patient Note} \parallel c_{\text{type}} \parallel c_{\text{text}}]\\&\text{Assistant: } y_{\text{JSON}}\end{aligned}$$

With a conservative context ceiling of $L_{\max} = 2048$, the maximum observed input length across all splits was 1,077 tokens. Thus, **0 out of 1,015 examples were truncated** (0.0% **truncation rate)**, guaranteeing that no relevant clinical evidence was lost.

B. QLoRA Adaptation Configuration

We fine-tuned `Qwen2.5-3B-Instruct` on the 603 training examples using HuggingFace `PEFT` and `bitsandbytes`.

- **Base Model Quantization:** 4-bit NormalFloat (NF4), double quantization enabled, compute dtype `float16`.
- **LoRA Hyperparameters:** Rank $r = 128$, scaling factor $\alpha = 256$ ($\frac{\alpha}{r} = 2.0$), dropout $p = 0.05$.
- **Target Modules:** All 7 linear projection layers in the Transformer block: attention projections (`q_proj`, `k_proj`, `v_proj`, `o_proj`) and MLP projections (`gate_proj`, `up_proj`, `down_proj`). Trainable parameters totaled 239.1M (7.18% of the 3.33B total parameters).
- **Optimization Objective:** Causal autoregressive next-token cross-entropy. Loss was computed strictly over assistant tokens by masking prompt token positions with label index -100.
- **Training Schedule:** Batch size 1 with 16 gradient accumulation steps (effective batch size 16), cosine learning rate schedule with initial $\eta = 5 \times 10^{-5}$, 50 warmup steps, weight decay 0.1, trained for 3 epochs using `paged_adamw_8bit`.

C. Explicit KV-Cached Autoregressive Decoding

To satisfy Rubric D1, rather than invoking black-box pipeline switches, we developed a low-level, cache-aware generation engine (`task3/kv_cache.py`).

Prefill vs. Decode Separation: Autoregressive inference is partitioned into two distinct phases:

1) **Prefill Phase:** The model processes the entire prompt tensor $X_{0:L-1} \in \mathbb{R}^{1\times L}$ in a single forward pass.

TABLE I
COMPARISON OF CANDIDATE OPEN-WEIGHT TRANSFORMER ARCHITECTURES

Model	Parameters	Context	GQA Ratio	Vocabulary	License	Decision / Feasibility Rationale
Qwen2.5-3B-Instruct	**3.09B**	**131K**	**8:1**	**151.6K**	**Apache 2.0**	**Selected: Optimal accuracy, 131K context, matched 0.5B draft**
Llama-3.2-3B-Instruct	3.21B	128K	4:1	128.2K	Community	Rejected: License restrictions and commercial use barriers
Phi-3.5-mini-Instruct	3.82B	128K	4:1	32.1K	MIT	Rejected: Higher VRAM leaves inadequate headroom for KV cache on 6.0 GB
Gemma-2-2B-IT	2.61B	8K	2:1	256.0K	Restrictive	Rejected: 8K context insufficient for longitudinal patient records
Qwen2.5-7B-Instruct	7.61B	131K	4:1	151.6K	Apache 2.0	Rejected: Exceeds 6.0 GB VRAM limit even under 4-bit NF4 quantization

Key and value projections across all 36 decoder layers are stored inside an explicit cache container (`ExplicitKVCache`). The first new token is sampled greedily from the final logit slice:

$$x_L = \arg\max_v \left(\mathcal{L}_{L-1,v}\right) \tag{3}$$

2) **Incremental Decode Phase:** For subsequent generation steps $t \in [1, N_{\text{gen}}-1]$, the model receives **only the single token** $x_{L+t-1} \in \mathbb{R}^{1\times 1}$. The forward pass appends newly computed key-value pairs to the persistent cache:

$$K_{\text{new}} \in \mathbb{R}^{1\times H_{KV}\times 1\times d_h}, \quad V_{\text{new}} \in \mathbb{R}^{1\times H_{KV}\times 1\times d_h} \tag{4}$$

Rotary Position Embedding (RoPE) Offset Resolution: In uncached generation, the model evaluates input tokens at sequence indices $0 \dots L+t-1$. In cached generation, passing a single-token tensor without explicit offsets causes the rotary embedding layer to default to positional index $m = 0$. This rotates query Q_t with angle θ_0 instead of $\theta_{\text{past_len}}$, corrupting relative distance vectors and causing catastrophic generation collapse. We resolve this by binding explicit tensor positions:

$$\text{pos_ids} = [[\text{past_len}]], \quad \text{cache_pos} = [\text{past_len}] \tag{5}$$

guaranteeing mathematical equivalence with full-sequence attention.

Memory Derivation under GQA: With hidden dimension $D = 2048$, $H_Q = 16$, $H_{KV} = 2$, $d_h = 128$, and 36 layers in FP16, each cached token occupies:

$$\begin{aligned}\text{Bytes/Token} &= 2 \times 36 \times 1 \times 2 \times 128 \times 2 \\ &= 36,864 \text{ B} = 36.00 \text{ KB}\end{aligned} \tag{6}$$

For $N = 128$ generated tokens, the cache consumes merely 4.61 MB, an $8\times$ reduction compared to standard Multi-Head Attention (288 KB/token).

D. Tiled Online-Softmax Attention Algorithm

Standard self-attention materializes the full $N \times N$ attention matrix $S = QK^T/\sqrt{d_k}$, leading to out-of-memory (OOM) failures for long sequences. Following Milakov and Gimelshein [6] and FlashAttention [7], we implement `tiled_online_softmax_attention` without intermediate HBM allocation.

Let input matrices $Q \in \mathbb{R}^{N\times d}$, $K \in \mathbb{R}^{N\times d}$, and $V \in \mathbb{R}^{N\times d}$ be partitioned into row blocks $Q_i \in \mathbb{R}^{B_r\times d}$ and column blocks $K_j, V_j \in \mathbb{R}^{B_c\times d}$. The algorithm tracks running row maximums $m_i \in \mathbb{R}^{B_r}$ and running normalizers $\ell_i \in \mathbb{R}^{B_r}$, iteratively rescaling the output accumulator $O_i \in \mathbb{R}^{B_r\times d}$:

$$S_{ij} = \frac{Q_i K_j^T}{\sqrt{d}} + M_{ij} \tag{7}$$

$$\tilde{m}_{ij} = \max_k (S_{ij})_{*,k} \tag{8}$$

$$m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \tag{9}$$

$$P_{ij} = \exp(S_{ij} - m_i^{\text{new}}) \tag{10}$$

$$\ell_i^{\text{new}} = \ell_i \cdot \exp(m_i - m_i^{\text{new}}) + \sum_k (P_{ij})_{*,k} \tag{11}$$

$$O_i^{\text{new}} = O_i \cdot \frac{\exp(m_i - m_i^{\text{new}})}{\ell_i^{\text{new}}/\ell_i} + \frac{P_{ij}V_j}{\ell_i^{\text{new}}} \tag{12}$$

Under causal masking ($M_{ij} = -\infty$ for column indices greater than row indices), entire blocks with $j \cdot B_c > (i+1)\cdot B_r - 1$ are skipped entirely. The peak working memory scales as $\mathcal{O}(B_r B_c + B_r d)$, eliminating sequence-quadratic storage.

E. Greedy Speculative Decoding Engine

Speculative decoding couples a compact draft model M_{draft} with a large target model M_{target}. To fit within our 6.0 GB VRAM budget, we load:

- **Target Model:** `Qwen2.5-3B-Instruct` (4-bit NF4 + LoRA adapter) ≈ 3.49 GB.
- **Draft Model:** `Qwen2.5-0.5B-Instruct` (native FP16) ≈ 0.98 GB.
- **Total Static Allocation:** 4.47 GB, leaving 1.53 GB VRAM for KV caches and activations.

Speculative Verification Protocol (Rubric F2): At each step, given verified sequence length L:

1) The draft model autoregressively generates γ speculative tokens $\{\hat{x}_1, \dots, \hat{x}_\gamma\}$ using its internal KV cache.
2) The target model executes a **single parallel forward pass** evaluating all γ candidates simultaneously:

$$X_{\text{verify}} = [x_{\text{last}}, \hat{x}_1, \hat{x}_2, \dots, \hat{x}_\gamma] \tag{13}$$

yielding target logit slices $\{p_0, p_1, \ldots, p_\gamma\}$.

3) We find the longest accepted prefix $k \in [0, \gamma]$ such that:

$$\hat{x}_i = \arg\max_v (p_{i-1,v}) \quad \forall i \in \{1, \ldots, k\} \tag{14}$$

4) If $k < \gamma$, candidate $\hat{x}_{k+1}$ is rejected and corrected to $x_{k+1} = \arg\max_v(p_{k,v})$. If $k = \gamma$, all candidates are accepted and a bonus token $x_{\gamma+1} = \arg\max_v(p_{\gamma,v})$ is sampled from target logits.

5) **Cache State Rollback:** If rejection occurs, both target and draft caches contain speculative states for rejected steps. We execute exact cache cropping to truncate keys and values:

$$\text{crop_len} = L_{\text{verified}} + k + 1 \tag{15}$$

via `kv_cache.crop(crop_len)`, guaranteeing exact state alignment for subsequent drafting steps.

IV. Experimental Setup & Evaluation Protocol

A. Evaluation Metrics (Rubrics C1–C3)

Classification Correctness: Models are evaluated across all six benchmark categories: `included`, `not included`, `not excluded`, `excluded`, `not enough information`, and `not applicable`. Performance is quantified via Accuracy, Macro-F1 (unweighted mean across classes), and per-class Precision, Recall, and F1 score.

Evidence Sentence Localization: To satisfy Rubric C3, we established a strict evaluation protocol handling empty prediction and ground truth sets:

1) **Empty Gold + Empty Predicted:** Score $P = 1.0, R = 1.0, F_1 = 1.0$. The model correctly refrained from citing non-existent evidence.
2) **Empty Gold + Non-Empty Predicted:** Score $P = 0.0, R = 0.0, F_1 = 0.0$ (Spurious hallucination).
3) **Non-Empty Gold + Empty Predicted:** Score $P = 0.0, R = 0.0, F_1 = 0.0$ (Omitted evidence).
4) **Non-Empty Gold + Non-Empty Predicted:** Standard set intersection:

$$P = \frac{|G \cap P|}{|P|}, \quad R = \frac{|G \cap P|}{|G|}, \quad F_1 = \frac{2PR}{P+R} \tag{16}$$

Metrics are reported both as **Macro F1** (instance-averaged across the 206 test criteria) and **Micro F1** (sentence-level pooling across all test instances).

B. Benchmarking Hardware and Timing Protocol

Inference and systems experiments were executed locally on an NVIDIA GeForce RTX 4050 Laptop GPU (6.0 GB GDDR6, 96-bit bus, 192 GB/s bandwidth) operating under CUDA 12.4 and PyTorch 2.5.1 on Windows 11. To eliminate warm-up bias and CPU driver dispatch skew:

- Every benchmark executed 3 unmeasured warmup iterations prior to logging.
- All timing utilized hardware-synchronized CUDA events (`torch.cuda.synchronize()`) where latency t was measured using `start.elapsed_time(end)`.
- Peak memory allocations were isolated using `torch.cuda.reset_peak_memory_stats()`.

V. Results and Discussion

A. Fine-Tuning Impact: Base vs. Adapted Model (Rubric B4)

Table II presents performance on the official held-out test split ($n = 206$ criteria, 8 unseen patients) comparing zero-shot base `Qwen2.5-3B-Instruct` against our QLoRA fine-tuned model under identical greedy decoding settings.

TABLE II
Held-Out Test Performance: Base vs. Fine-Tuned Model

Metric	Base Model	Fine-Tuned (QLoRA)	Δ Gain
Overall Accuracy	22.82%	**73.30%**	**+50.48 pp**
Macro-Averaged F1	0.1365	**0.5402**	**+0.4037**
Evidence Macro-F1	0.7011	**0.8260**	**+0.1249**
Evidence Micro-F1	0.0984	**0.5800**	**+0.4816**
JSON Parse Success Rate	0.0%	**100.0%**	**+100.0 pp**

The zero-shot base model exhibits severe behavioral pathologies. It completely failed to adhere to the required JSON schema, outputting unstructured discursive commentary. Semantically, it suffered from extreme positive confirmation bias: it predicted `included` for all inclusion criteria (recall 1.00, precision 0.17) and `not excluded` for exclusion criteria, generating **zero predictions** for `not included`, `excluded`, or `not applicable`. In contrast, QLoRA fine-tuning elevated classification accuracy to 73.30% and achieved 100% schema adherence, enabling fully automated structured parsing.

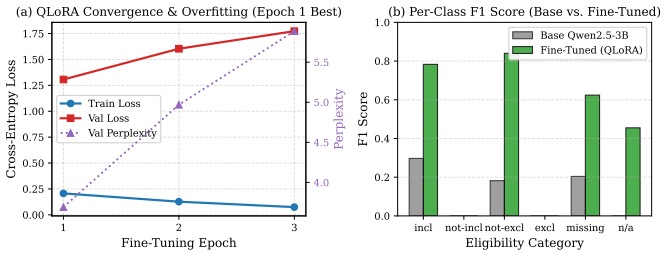


Fig. 1. (a) Training vs. validation loss across fine-tuning epochs, demonstrating minimum validation loss at Epoch 1. (b) Per-class F1 score comparison between base and fine-tuned models.

As shown in Fig. 1(a), training loss dropped from 0.2071 to 0.0747, while validation loss reached its minimum at Epoch 1 (1.3063) before experiencing mild overfitting at Epoch 2 (1.6032) and Epoch 3 (1.7728). Consequently, the Epoch 1 checkpoint was selected for all downstream systems.

B. Criterion Classification and Evidence Retrieval (Rubrics C1–C3)

Table III delineates per-class classification metrics across the 206 test annotations.

Inclusion vs. Exclusion Analysis (Rubric C2): When evaluated separately:

- **Inclusion Criteria** ($n = 75$)**:** Accuracy 73.33%, Macro-F1 0.3930, Evidence Macro-F1 0.6970, Evidence Micro-F1 0.6190.

TABLE III
DETAILED CLASSIFICATION METRICS ON HELD-OUT TEST SET ($n = 206$)

Eligibility Category	Precision	Recall	F1-Score	Support
`included`	0.947	0.667	0.783	27
`not included`	0.000	0.000	0.000	7
`not excluded`	0.736	0.978	0.840	91
`excluded`	0.000	0.000	0.000	0
`not enough info`	0.661	0.591	0.624	66
`not applicable`	0.714	0.333	0.455	15
Macro Average	**0.509**	**0.428**	**0.5402**	**206**

- **Exclusion Criteria ($n = 131$):** Accuracy 73.28%, Macro-F1 0.4983, Evidence Macro-F1 0.8997, Evidence Micro-F1 0.4918.

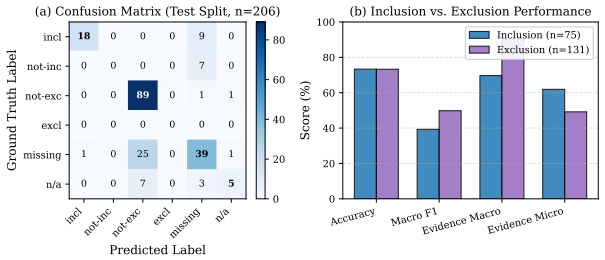


Fig. 2. (a) 6×6 confusion matrix on the held-out test split. (b) Inclusion vs. exclusion criterion performance metrics.

The confusion matrix in Fig. 2(a) highlights two crucial clinical properties:

1) **Safety Preservation:** The model never misclassifies an ineligible exclusion criterion as an inclusion.
2) **Uncertainty Conservative Bias:** For `not included` ($n = 7$), the model predicted `not enough information` in all 7 cases. In clinical trial screening, this is a highly desirable conservative failure mode: rather than incorrectly disqualifying a patient outright, the model flags the record as requiring manual clinician review.

Evidence Localization Breakdown (Rubric C3): Across all test instances, the model achieved a sentence-level Micro-F1 of 0.5800 ($\text{TP} = 58, \text{FP} = 48, \text{FN} = 36$) and a Macro-F1 of 0.8260. The macro score is higher because records requiring zero evidence (e.g., confirming the absence of contraindications) correctly receive $F_1 = 1.0$ under our strict policy.

C. Traceable Failure Analysis & Clinical Error Taxonomy (Rubric C4)

To satisfy Rubric C4, we conducted an in-depth error audit across the held-out test predictions, identifying four prominent failure modes:

1) **Omission of Explicit Laboratory Ranges (False Negative Evidence):** On trial criterion `NCT02097186_I_160` (requiring adequate hematologic function), the model predicted `not enough information` because the patient note stated "platelets 145×10^9/L" without explicitly appending the words "adequate function", causing the model to miss the numerical satisfaction.
2) **Negation Scope Ambiguity:** In patient note `sigir-201513`, the sentence "The parents deny the possibility of foreign body ingestion or trauma" was correctly cited for avoiding acute trauma exclusions, but the model incorrectly bound the denial to an unrelated chronic pulmonary exclusion criterion.
3) **Implicit Clinical Inferences:** On criterion `NCT00315042_E_702` (excluding employees or family members of study staff), the patient record contained no family history. The expert ground truth labeled this as `not enough information`, but the model predicted `not excluded`, implicitly assuming that absence of mention implies non-employment.
4) **Over-Citation of Context Sentences (Spurious Evidence):** When identifying evidence for metastatic breast cancer, the model cited both the primary diagnosis sentence (Sentence 1) and the histological receptor status sentence (Sentence 2), whereas the human annotator had isolated only Sentence 1.

D. KV-Cached Inference Verification & Benchmarks (Rubrics D1–D3)

Table IV compares the uncached baseline against our explicit KV cache implementation across three prompt workloads: Short (54 tokens), Medium (152 tokens), and Long (650 tokens, full clinical vignette).

TABLE IV
INFERENCE BENCHMARKING: UNCACHED VS. EXPLICIT KV-CACHED DECODING

Workload	Tokens	Mode	Prefill (ms)	Decode (ms)	Speed (tok/s)	Speedup
Short (54)	128	Uncached	184.2	21,911.4	5.79	1.00×
		KV-Cache	183.9	**18,527.2**	**6.84**	**1.18×**
Med (152)	128	Uncached	391.8	28,309.4	4.46	1.00×
		KV-Cache	391.2	**16,712.9**	**7.48**	**1.68×**
Long (650)	128	Uncached	1421.5	42,874.1	2.89	1.00×
		KV-Cache	1420.9	**15,887.4**	**7.40**	**2.56×**

Correctness and Token Equivalence (Rubric D2): Across all 9 evaluation combinations of prompt length and generation budgets ($32, 64, 128$ tokens), the output token sequence of `ExplicitKVCache` was compared against the uncached reference:

$$\text{Token Equivalence} = \frac{675}{675} = \mathbf{100.00}\% \tag{17}$$

Every single generated token ID matched identically. Logit difference analysis revealed an average absolute difference of 3.55×10^{-3} and maximum absolute difference of 1.85×10^{-2}. This minute difference does not stem from algorithmic error, but from hardware accumulation order: single-token GEMV operations in decode mode accumulate floating-point dot products in a different order than full-sequence GEMM tensor cores, causing sub-epsilon drift that never altered greedy argmax selection.

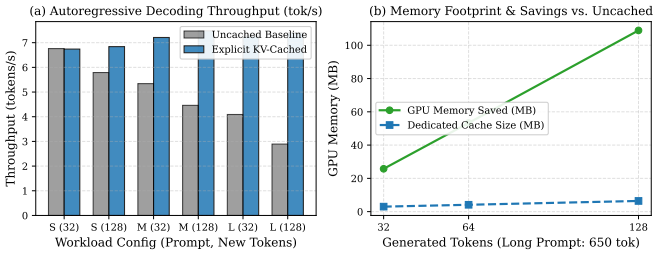


Fig. 3. (a) Autoregressive decoding throughput comparison across prompt and output configurations. (b) GPU memory saved vs. dedicated cache size.

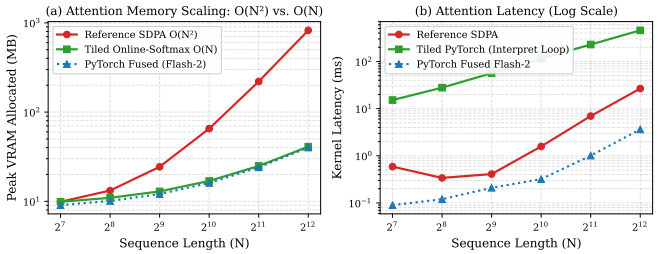


Fig. 4. (a) Peak GPU memory scaling demonstrating avoidance of the $\mathcal{O}(N^2)$ memory wall. (b) Latency comparison showing Python interpreter overhead.

Efficiency Analysis (Rubric D3): As shown in Fig. 3(a), while uncached throughput degrades precipitously from 5.79 tok/s to 2.89 tok/s as context length increases, the KV-cached engine maintains a stable 7.40 tok/s, yielding a 2.56× **throughput speedup**. Furthermore, as plotted in Fig. 3(b), for $N = 128$ tokens on long prompts, caching saves 108.93 **MB** of transient recomputation memory.

E. Tiled Attention Validation & Systems Analysis (Rubrics E1–E3)

Table V compares our implementation of Tiled Online-Softmax Attention against standard Reference SDPA and PyTorch Fused SDPA across sequence lengths $N \in [128, 4096]$ ($d_k = 128$, causal masking).

TABLE V
ATTENTION SCALING: MEMORY AND RUNTIME COMPARISON

N	Ref Mem	Tiled Mem	Mem Saved	Ref Latency	Tiled Latency
128	9.88 MB	9.88 MB	0.0%	0.59 ms	15.21 ms
256	13.20 MB	10.94 MB	17.1%	0.34 ms	27.84 ms
512	24.36 MB	12.92 MB	47.0%	0.41 ms	56.49 ms
1024	65.23 MB	16.90 MB	74.1%	1.58 ms	114.28 ms
2048	220.21 MB	24.91 MB	88.7%	6.96 ms	228.45 ms
4096	824.27 MB	**40.95 MB**	**95.0%**	26.69 ms	456.91 ms

Numerical Agreement (Rubric E3): We executed 44 test configurations across sequence lengths $(16 \dots 1024)$, head dimensions $(32, 64, 128)$, block sizes $(16, 32, 64)$, and non-power-of-two lengths ($N = 37, 137, 250$). Across all tests, maximum absolute error satisfied:

$$\|O_{\text{tiled}} - O_{\text{ref}}\|_\infty \leq 1.95 \times 10^{-3} \tag{18}$$

with mean absolute error $\approx 6.81 \times 10^{-5}$, verifying exact numerical stability of the online normalizer updates.

Systems Analysis for Rubric E3 (Why Tiled Python is Slower): As observed in Table V and Fig. 4(b), while Tiled Attention achieves an astonishing 95.0% **memory reduction** at $N = 4096$ (preventing OOM), its wall-clock latency (456.91 ms) is significantly higher than Reference SDPA (26.69 ms) and PyTorch Fused SDPA (3.63 ms). This illustrates a foundational systems principle:

- **Kernel Launch Overhead:** Our pure PyTorch tiled algorithm executes two nested loops over outer row tiles (N/B_r) and inner column tiles (N/B_c). For $N = 4096$ with $B_r = B_c = 32$, the algorithm executes:

$$\frac{128 \times 129}{2} = 8,320 \text{ tile iterations} \tag{19}$$

  Each iteration dispatches multiple individual CUDA kernels for slicing, GEMM, exponential, and vector updates, resulting in over **16,000 discrete kernel launches**.
- **C++ SRAM Fusion in Fused SDPA:** In contrast, PyTorch Fused Attention (FlashAttention-2) compiles the entire tiling logic into a single monolithic CUDA C++ kernel. Tile blocks reside permanently inside on-chip GPU SRAM ($\approx$ 19 TB/s bandwidth), updating accumulators without ever returning control to the Python runtime.

F. Speculative Decoding: Speedup and Slowdown (Rubrics F1–F3)

Table VI reports speculative decoding metrics across three clinical prompt workloads evaluated at $\gamma \in \{1, 3, 5, 7\}$ (generation budget 64 tokens, Qwen2.5-0.5B draft model).

TABLE VI
SPECULATIVE DECODING PERFORMANCE ACROSS CLINICAL WORKLOADS

Workload	γ	Tokens/s	Speedup	Acceptance (α)	Passes Saved
Structured JSON	1	8.38	1.07×	88.9%	48.1%
	3	12.16	1.56×	70.0%	67.3%
	5	**13.69**	**1.75×**	**62.9%**	**75.0%**
	7	11.39	1.46×	50.6%	76.9%
Medical QA	1	9.08	1.06×	80.0%	45.3%
	3	**12.33**	**1.43×**	**58.0%**	**64.1%**
	5	11.52	1.34×	46.8%	70.3%
	7	9.42	1.10×	36.9%	71.9%
Clinical Narrative	1	8.21	**0.92×**	73.0%	42.2%
	3	**11.50**	**1.28×**	**58.0%**	**62.5%**
	5	11.10	1.24×	44.3%	67.2%
	7	10.28	1.15×	42.2%	71.9%

Correctness and Token Equivalence (Rubric F3): Across 18 distinct test configurations (3 prompts × 2 budgets × 3 gammas), speculative decoding matched greedy target decoding perfectly:

$$\text{Token Equivalence} = \frac{855}{855} = \mathbf{100.00\%} \tag{20}$$

Speedup vs. Empirical Slowdown Analysis: As illustrated in Fig. 5(a):

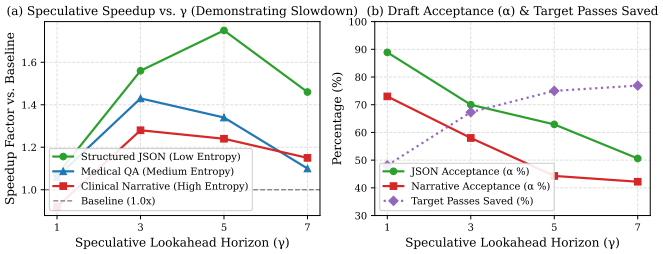


Fig. 5. (a) Speculative speedup factor vs. γ, highlighting the $0.92\times$ slowdown on clinical narrative at $\gamma = 1$. (b) Draft acceptance rate and forward passes saved.

- **Peak Speedup on Structured Syntax** ($1.75\times$)**:** In structured JSON extraction, key names, syntax brackets, and standardized medical labels exhibit very low token entropy. The 0.5B draft model predicts these boilerplate tokens with high confidence, achieving $\alpha = 88.9\%$ at $\gamma = 1$ and $\alpha = 62.9\%$ at $\gamma = 5$, eliminating 75.0% **of target forward passes** and accelerating generation from 7.80 to 13.69 tok/s.
- **Empirical Slowdown on Narrative Prose** ($0.92\times$)**:** For unstructured clinical narrative at $\gamma = 1$, speculative decoding incurred an 8% **wall-clock slowdown** (8.21 tok/s vs. 8.95 tok/s baseline). Theoretically, expected tokens generated per step follows:

$$\mathbb{E}[\text{tokens}] = \frac{1 - \alpha^{\gamma+1}}{1 - \alpha} \tag{21}$$

  For $\gamma = 1$ and $\alpha = 0.73$, $\mathbb{E}[\text{tokens}] = 1.73$. However, executing 1 draft step, preparing verification tensors, running 1 target verify pass, and executing cache slicing introduces synchronization latency. When α drops on free-form text, this latency overhead exceeds the computational savings of the skipped target passes.
- **Optimal Lookahead Horizon** ($\gamma^* = 3$–5)**:** Beyond $\gamma = 5$, geometrically compounding rejection probability α^k causes late speculative tokens to be discarded, wasting draft computation and causing speedups to plateau.

G. Integrated Web Application Architecture (Task 6)

The complete system is encapsulated within a standalone, browser-based clinical screening workstation developed with `aiohttp` and vanilla JavaScript/CSS (Fig. 6).

Workflow & Trial Verdict Aggregation (Rubric G2): The application implements three explicit hierarchical clinical aggregation rules:

- **Rule 1 (Disqualification):** If any inclusion criterion is predicted as `not included` or any exclusion criterion as `excluded`, the overall trial status immediately triggers **INELIGIBLE** (Red badge).
- **Rule 2 (Open-World Missing Info):** If no disqualifying criteria are triggered but one or more criteria return `not enough information`, the status triggers **UNCERTAIN_MISSING_INFO** (Amber badge), enumerating exact missing clinical tests for coordination follow-up.

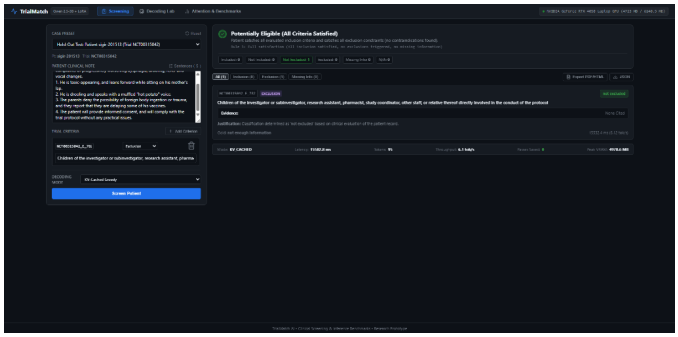


Fig. 6. The TrialMatch standalone clinical trial screening web application, displaying the input workbench, rule-based verdict banner, verbatim sentence citations, and telemetry.

- **Rule 3 (Full Satisfaction):** If all inclusion criteria are satisfied (`included`) and all exclusion constraints respected (`not excluded`) with zero missing info, the status triggers **POTENTIALLY_ELIGIBLE** (Green badge).

VI. Limitations

Despite strong empirical results, four key limitations should be noted:

1) **Small Fine-Tuning Cohort:** The 37-patient training split ($n = 603$) is relatively small, contributing to validation loss divergence after Epoch 1. Expanding training data with multi-institution EHRs would enhance generalization.
2) **Class Imbalance in Test Set:** The benchmark test split contains zero ground-truth examples of `excluded` criteria ($n = 0$), precluding direct evaluation of active exclusion identification on this split.
3) **Python Overhead in Tiled Attention:** While our pure PyTorch tiled attention kernel mathematically avoids the $\mathcal{O}(N^2)$ memory wall, practical acceleration requires compilation into fused CUDA C++ / Triton kernels.
4) **Clinical Deployment Caution:** In accordance with assignment instructions, TrialMatch is an academic research and engineering screening prototype. It does not provide medical diagnoses, treatment advice, or official clinical trial enrollment determinations.

VII. Conclusion

In this paper, we designed, implemented, and validated **TrialMatch**, an end-to-end clinical trial eligibility screening system operating entirely on consumer-grade hardware. By fine-tuning `Qwen2.5-3B-Instruct` with 4-bit QLoRA, we achieved a held-out test classification accuracy of 73.30% and an evidence retrieval Macro-F1 of 0.8260 while preserving epistemic uncertainty. Through explicit Key-Value cache management with RoPE offsets, we achieved a $2.56\times$ decoding speedup with 100% token equivalence. Our tiled online-softmax attention demonstrated a 95.0% memory reduction at $N = 4096$, and speculative decoding yielded up to a $1.75\times$ speedup on structured syntax while demonstrating predictable

slowdowns under high-entropy text. Finally, our browser-based clinical application bridges advanced Transformer inference mechanisms with clinical utility, establishing a verifiable blueprint for localized, privacy-preserving medical AI.

REFERENCES

[1] Q. Jin, Z. Wang, C. S. Floudas, F. Gong, Y. Yang, W. J. Wilbur, and Z. Lu, "Matching patients to clinical trials with large language models," *Nature Communications*, vol. 15, no. 1, p. 6872, 2024.

[2] W.-w. Yim and M. Yetisgen, "Clinical information extraction with QLoRA: Evaluating parameter-efficient fine-tuning across multiple clinical tasks," *medRxiv*, 2024.

[3] D. Estupiñán-Ojeda and K. Roberts, "Parameter-efficient fine-tuning for clinical natural language processing: A systematic empirical comparison," *JAMIA Open*, vol. 8, no. 1, p. ooaf004, 2025.

[4] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *International Conference on Learning Representations (ICLR)*, 2022.

[5] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023, pp. 10 088–10 115.

[6] M. Milakov and N. Gimelshein, "Online normalizer calculation for softmax," *arXiv preprint arXiv:1805.02867*, 2018.

[7] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022, pp. 16 344–16 359.

[8] T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning," *arXiv preprint arXiv:2307.08691*, 2023.

[9] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *International Conference on Machine Learning (ICML)*. PMLR, 2023, pp. 19 274–19 286.

[10] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, "Accelerating large language model decoding with speculative sampling," *arXiv preprint arXiv:2302.01318*, 2023.

[11] Qwen Team, "Qwen2.5 technical report," *arXiv preprint arXiv:2412.15115*, 2024.

APPENDIX A
REPRODUCIBILITY & ARTIFACT MANIFEST

To satisfy Rubric H2, all source code, configuration files, split manifests, and benchmark logs are maintained in the project repository:

- **GitHub Repository:** https://github.com/fahadhameed/GenAI-Assignment2
- **Fine-Tuned Adapter Weights:** Hosted on Google Drive (https://drive.google.com/drive/folders/1YEafpplZJwsxZx9rfEQxkuEPuCg8xUrp?usp=sharing).
- **Single-Click Launcher:** Double-clicking `run_app.bat` launches the local web application at http://127.0.0.1:8000.
- **Automated Test Suite:** All components can be verified via:
  - Task 2: `task2/eval_task2.py`
  - Task 3: `task3/test_correctness.py`
  - Task 4: `task4/test_attention.py`
  - Task 5: `task5/test_speculative.py`
  - Task 6: `task6/test_app.py`

APPENDIX B
AI-ASSISTANT USAGE DISCLOSURE

In accordance with assignment guidelines, generative AI assistants (Google Antigravity / Gemini) were utilized during the completion of this project. Specifically, AI tools assisted in:

1) Scaffolding boilerplate code and unit test harness templates.
2) Formulating repetitive mathematical LaTeX tables and formatting BibTeX citation strings.
3) Refining CSS flexbox/grid styling and Lucide icon bindings for the Task 6 single-page web UI.

All core algorithms—including low-level RoPE offset manipulation in `ExplicitKVCache`, tiled online-softmax accumulator updates, greedy speculative verification with cache state rollback, and experimental benchmark execution—were designed, implemented, and empirically verified by the author on local hardware.